



리눅스 서버 사용 및 관리 설명서

version 1.01

A. Uthor

관리자 A. Nother

부관리자 T. Heother

Computer Science & Engineering

Engineering

Seoul National University

2026. 1. 17.

이 설명서는 비영리적 목적에 한해 제한 없이 배포할 수 있습니다.



서울대학교
SEOUL NATIONAL UNIVERSITY

Copyright ©2026 Seoul National University

[HTTPS://EN.SNU.AC.KR/INDEX.HTML](https://en.snu.ac.kr/index.html)

Compiled at February 23, 2026

To The Researchers of Seoul National University

Calculation is the best way to prove.

Acknowledgements

acknowledgements

Preface

현대의 이공계 연구와 산업 현장은 더 이상 이론적 이해나 실험적 숙련도만으로 완결되지 않는다. 수치해석, 시뮬레이션, 데이터 분석, 자동화, 인공지능 및 고성능 계산(HPC)에 이르기까지, 코딩과 소프트웨어 개발 능력은 모든 이공계 분야에서 사실상 필수적인 연구 언어이자 사고 도구가 되었다. 물리학·화학·생명과학·재료과학·기계공학·전산학 등 학문적 경계를 막론하고, 연구의 설계, 수행, 검증, 재현성 확보의 핵심 단계마다 컴퓨터 기반 계산과 프로그래밍이 깊숙이 관여하고 있다.

그러나 이러한 중요성에도 불구하고, 많은 이공계 학생들은 코딩 그 자체보다도 개발 및 계산 환경을 구축하는 과정에서 큰 장벽을 경험한다. 운영체제별 차이, 컴파일러와 라이브러리의 의존성 문제, 패키지 충돌, 환경 변수 설정, 원격 서버 및 클러스터 사용, 버전 관리 시스템의 개념 등은 정규 교과과정에서 체계적으로 다루어지기 어렵다. 그 결과, 학생들은 “코드는 맞는데 실행이 되지 않는” 상황, “남의 컴퓨터에서는 되는데 내 환경에서는 안 되는” 문제, 혹은 “서버에서 무엇이 돌아가고 있는지 이해하지 못한 채 명령어를 복사·붙여넣기만 하는” 상태에 놓이기 쉽다. 이러한 환경적 불확실성은 학습 효율을 떨어뜨릴 뿐만 아니라, 연구 재현성과 협업 능력에도 부정적인 영향을 미친다.

이 문서는 바로 이러한 문제의식에서 출발한다. 본 문서는 특정 언어나 단일 소프트웨어의 사용법을 나열하는 안내서가 아니라, 이공계 학생이 연구와 개발을 수행하기 위해 반드시 이해해야 할 컴퓨터 및 서버 환경의 구조와 원리를 체계적으로 정리하는 것을 목표로 한다. 개발을 위한 Linux 환경 설정, 패키지 관리와 가상 환경의 개념, 컴파일과 실행의 흐름, 버전 관리 등을 단계적으로 다룸으로써, 독자가 단순한 사용자 수준을 넘어 자신의 계산 환경을 스스로 설계·관리할 수 있는 역량을 갖추도록 돕고자 한다.

이 문서를 통해 독자는 다음과 같은 것을 얻을 수 있을 것이다. 첫째, 개발 환경에서 발생하는 다양한 오류와 혼란을 “요령”이 아닌 구조적 이해를 바탕으로 진단하고 해결하는 능력. 둘째, 개인 컴퓨터에서 시작해 연구실 서버, 나아가 대규모 계산 자원까지 확장 가능한 일관된 작업 환경을 구축하는 사고방식. 셋째, 협업과 장기 연구에서 필수적인 재현 가능하고 유지보수 가능한 계산 환경을 설계하는 감각.

궁극적으로 이 문서는 코딩을 “해야만 하는 기술”이 아니라, 이공계 연구를 사고하고 확장하는 보편적 도구로 받아들이는 데 필요한 기초 체력을 제공하고자 한다. 환경 설정에 대한 이해는 눈에 잘 띄지 않지만, 연구 생산성과 독립성을 결정짓는 핵심 요소이다. 이 문서가 독자가 보다 안정적이고 자유로운 계산 환경 위에서 연구와 개발에 집중할 수 있는 발판이 되기를 바란다.

익명의 누군가

2026년 1월 17일, 관악사에서.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Windows vs. Linux	1
1.3	WSL	2
2	Linux	3
2.1	파일 관리	3
3	Python Environments	4
3.1	주의사항	4
4	Compile Environments	5
5	Git	6
6	Server	7
	Appendix A Contributors	8

List of Figures

List of Tables

List of Abbreviations

GUI Graphic User Interface

CLI Command Line Interface

Introduction

적당한 첫 문장

1.1 | Motivation

현대 이공계 연구에서 컴퓨터를 이용한 계산은 절대로 피할 수 없는 요소가 되었다. 특히, 대규모 데이터 처리, 복잡한 수치 계산, 시뮬레이션 등은 고성능 컴퓨팅 환경에서 수행되는 경우가 많다. 여러 대학에서도 기초 교양으로 컴퓨터 과목을 운영하고 있고, 컴퓨터공학과가 아닌 전공에서도 수업에서 프로그래밍이 요구되는 경우가 점점 늘어나고 있다. 그러나 대부분의 경우, 프로그래밍 자체는 가르쳐 주더라도 그 프로그래밍을 위한 적절한 환경 설정 방법을 잘 알려주지 않는다. 필자의 경우에도 새로 연구실에 들어온 학생이 리눅스 로컬 및 서버 환경 설정에서 큰 어려움을 겪거나, 학과 후배들이 프로그래밍 과제를 할 때 환경 설정부터 막히는 경우를 자주 보아왔다. 이에, 본 자료에서는 프로그래밍을 위한 환경 설정 방법을 가르쳐 주고자 한다. 본 자료에서는 독자가 컴퓨터에 대한 기초적인 지식은 가지고 있다고 가정한다.

1.2 | Windows vs. Linux

컴퓨터 사용자가 컴퓨터를 쉽게 사용하기 위해서는 하드웨어와 소프트웨어 전반을 관리하고, 여러 서비스들을 제공하는 운영체제 (Operating System, OS)가 필요하다. 대표적인 운영체제로는 Microsoft사에서 제공하는 Windows와 Apple사에서 제공하는 macOS, 그리고 오픈 소스로 제공되는 Linux 계열 운영체제들이 있다.

일반적으로는 Windows가 가장 널리 사용되는 운영체제지만, 개발 환경에 있어서 Windows는 그리 적합하지 않다.

이러한 이유 때문에 프로그래밍을 할 때는 Windows보다 Linux 계열 운영체제를 사용하는 것이 일반적이다.

대표적인 Linux 계열 운영체제로는 Ubuntu Linux가 있으며, 윈도우와 달리 무료이며 공개 소스이다. 서버에 일반적으로 이용하는 Ubuntu Server는 GUI가 없는 Ubuntu로, 마우스 없이 명령줄만으로 제어하는 CLI 환경이다. 윈도우의 명령 프롬프트(cmd)와 유사하다. GUI를 사용하고 싶으면 Ubuntu Desktop을 설치하면 된다. 그러나, 슈퍼컴퓨터와 같은 고성능 컴퓨팅 환경에서는 대부분 CLI 환경을 사용하기 때문에, 추후 이러한 분야에 관심이 있다면 CLI 환경에 익숙해지는 것이 좋다.

1.3 | WSL

앞서 언급했듯이, 프로그래밍을 위해서는 Linux 계열 운영체제를 사용하는 것이 더 편리하다. 그러나 대부분의 컴퓨터에는 Windows가 설치되어 있으며, Windows에서만 사용할 수 있는 응용 프로그램들도 많다. 대표적으로 카카오톡을 Linux에서 사용하는 것은 굉장히 불편하다. 또한 컴퓨터에 익숙하지 않은 사람이 운영체제를 직접 건드리는 것은 위험할 수 있다. 따라서 Windows에서 Linux로 완전히 갈아타는 것은 쉽지 않다.

이를 해결할 수 있는 방법이 바로 Windows Subsystem for Linux (WSL)이다. WSL은 Windows 10과 Windows 11에서 사용할 수 있는 기능으로, Windows 내에서 Linux 환경을 실행할 수 있게 해준다. WSL을 사용하면 Windows와 Linux를 동시에 사용할 수 있으며, 파일 시스템도 공유할 수 있다. 또한 WSL은 가상 머신보다 훨씬 가볍고 빠르기 때문에, 개발자들에게 매우 인기가 있다.

WSL을 설치하는 방법은 간단하다 (<https://learn.microsoft.com/ko-kr/windows/wsl/install>). 이전에 WSL을 설치한 적이 없다면, Windows Powershell을 관리자 권한으로 실행한 후, 다음 명령어를 입력하여 WSL을 설치할 수 있다.

```
C:\Windows\system32> wsl --install
```

위의 명령어를 입력하고 설치가 완료될 때까지 기다리면, WSL이 자동으로 설치된다. 설치가 완료되면, 컴퓨터를 다시 시작하면 된다. 이제 WSL이라는 응용 프로그램이 생겼을 것이고 이를 실행할 수 있을 것이다. 이를 실행하면 cmd와 비슷한 창이 나타날 것인데, 이 창이 바로 CLI 환경의 Ubuntu Linux이다.

WSL이 좋은 점은 파일 시스템을 공유하기 때문에 Windows에서 Linux 파일에 접근할 수 있고, 그 반대도 가능하다는 것이다. WSL 내에서 Windows의 C 드라이브는 `/mnt/c`로 마운트되어 있고, WSL의 파일 역시 Windows의 파일 탐색기에서 접근할 수 있다.

여기까지 성공적으로 진행되었으면 이제 Linux 환경에서 프로그래밍을 할 수 있는 준비가 된 것이다. 다음 장에서는 Linux 환경에서 프로그래밍을 하기 위한 기본적인 명령어와 도구들을 소개할 것이다.

Linux

이 장은 Linux 명령어, bash script, .bashrc 설정, WSL과 VSCode 연결 및 설정 등에 대해 다룬다.

2.1 | 파일 관리

2.1.1 | 절대경로와 상대경로

절대경로는 `/home/user/dir1/dir2` 와 같은 것이다. 상대경로는 현재 내 위치에서 시작하는 경로다. 즉, 내가 지금 `/home/user/dir1` 에 있다면 여기서 `/home/user/dir1/dir2` 의 상대경로는 그냥 `dir2` 다.

2.1.2 | 디렉토리 및 탐색

`cd` 명령어를 사용해서 디렉터를 탐색할 수 있다.

`..` 은 상위 디렉토리, `-` 은 이전 디렉토리, `~` 는 홈 디렉토리를 의미한다. `cd <dir>` 라 입력하면 해당 디렉토리로 이동된다. 디렉토리를 표현할 때 절대경로, 상대경로 둘 다 가능하며, 실행시 주소쪽 값이 변경된다.

```
userid:~$ cd dir1
userid:~/dir1$ cd /home/user/dir1
userid:/home/user/dir1$ cd ..
userid:/home/user$ cd -
userid:/home/user/dir1$ cd ~
userid:~$
```

Python Environments

이 장은 Python 기반의 프로그래밍을 위한 기본 연구 환경(conda와 pip 설치, 가상환경, 패키지 설치)을 다룬다.

3.1 | 주의사항

- **sudo reboot**은 그 어떠한 경우에도 사용하면 안 된다. 새로운 프로그램을 설치하는 등 작업 도중 블로그 글을 따라쓰게 될 수 있는데, 그중에 reboot 명령어가 있으면 건너뛰자. 타인의 프로세스가 비정상적으로 종료될 위험이 크다.
- 모든 사용자는 반드시 가상환경을 구성하고, 그 가상환경 내에 여러가지 연구에 필요한 프로그램들을 설치해 사용해야 한다. 이렇게 하면 관리자 권한 없이 원하는 대로 프로그램의 버전이나 설정을 바꿀 수 있다.

Compile Environments

이 장은 컴파일 언어 기반의 프로그래밍을 위한 기본 연구 환경을 다룬다. gcc와 gfortran 설치, Makefile, CMake 사용법 등을 포함한다.

Git

이 장은 Git과 GitHub을 이용한 버전 관리 방법을 다룬다.

Server

이 장은 원격 서버 연결 방법, VSCode 연결 방법, Slurm scheduler 사용법 등을 다룬다.

Contributors

김도영 dobby@snu.ac.kr 홍승주 sjhong6230@snu.ac.kr